GNN + Conformal Prediction + SVGD

Multi-Robot Robustness-Aware Trajectory Planning

Meeting Brief and Presentation Notes

Prepared 15 October 2026

> **Main question:** Can a trained New GATv2 model, calibrated with conformal prediction, provide a useful robustness signal inside an SVGD trajectory planner—accurately and quickly enough to improve multi-robot trajectories?

1. The 30-second explanation

The professor asked for two connected studies:

1. Improve dataset construction and batching: avoid unnecessary padding; if padding is required by JIT, make it efficient.
2. Integrate the GNN and conformal prediction (CP) into the cost used by SVGD. If SVGD is ineffective or too slow, investigate MPPI or MBD.

The current system has an offline learning stage and an online planning stage:

Labeled graphs	→	Train New GATv2	→	Saved checkpoint
Calibration graphs	→	Fit CP correction	→	Calibrated scorer
Candidate team trajectories	→	Graphs at future steps	→	GATv2 + CP cost
Trajectory costs	→	SVGD updates candidates	→	Selected joint trajectory

Current result: In one synthetic 10-robot scenario, the simple planner selected a trajectory with minimum exact robustness 1, the soft-degree planner selected 0, and the GNN+CP planner selected 2. GNN+CP took 2.276 s versus 0.333 s for the simple planner. This demonstrates integration, but it is not yet evidence of general performance.

Theoretical correction: the demo uses a robustness threshold of 2 only as a placeholder. If the system uses W-MSR under an F-local adversary model, the usual sufficient scalar threshold is r = 2F + 1. For F = 1, the experiment should target r = 3.

2. What was changed in the training pipeline

Historical representation

Every graph was expanded to 50 node slots and 2,500 edge slots before training. Batches were assembled from graph objects with `Batch.from_data_list()`. This caused large padded tensors and additional host-side assembly work.

Current “New” representation

Each graph is stored in CPU memory with only its real nodes and edges. When a mini-batch is formed, the real graphs are concatenated. Limited padding is then added to obtain reusable JAX shapes:

- node capacity is bounded for the batch and includes a disconnected dummy node;
- edge capacity uses power-of-two buckets to reduce the number of JIT compilations;
- padding edges point only to the dummy node;
- padding nodes receive a batch ID outside the real pooling range;
- a background thread prepares the next host batch while the device processes the current batch.

Padding is still used because JAX normally compiles for array shape. The improvement is that padding occurs at batch construction, with bounded reusable shapes, rather than expanding every stored graph to the global maximum.

Matched GATv2 comparison	Historical-style padded representation	New compact representation
Dataset and split	Same training_data.json, 178,220 train / 19,803 validation, batch size 32	
Steady training time per epoch	about 94.4 s	about 16.0 s
Independent test accuracy	93%	88%
Interpretation	Higher accuracy in this run, much slower	About 5.9× faster, lower accuracy in this run

This matched comparison is the useful representation study. Historical scripts trained on only an 80/20 split of testing_data.json and reported the early-stopping subset, so those historical metrics are not directly comparable with the integrated runs.

3. What is in training_data.json?

The file contains **198,023 independent labeled graphs**. It does not contain robot coordinates or trajectories.

Dataset property	Observed value
Number of graphs	198,023
Nodes per graph	2–49
Directed edge entries per graph	0–2,124; mean about 356
Robustness labels	0–23
Robustness-zero graphs	29,369
Near-integer labels normalized when loaded	838

A typical record contains:

```
{
  "nodes": 29,
  "edge_index": [[sources...], [targets...]],
  "degree": [degree_of_node_0, ...],
  "robustness": 10.0,
  "membership": [...],
  "r_robustness": 10.0,
  "lambda2": 12.97
}
```

The New GATv2 uses only:

- `degree` to form node features `[degree, 1]`;
- `edge_index` to describe graph connectivity;
- `robustness` as the class label.

> **Distribution question:** the training file does not explain how graphs were generated from physical positions. The planner currently creates an edge whenever robot distance is below a communication radius. We must verify that this graph-generation rule is consistent with the training-data distribution before treating planner predictions as reliable.

4. GNN input and output

For one graph, each robot is one node. The model receives:

Input	Meaning
Node features	[number of neighbors, 1] for each robot
edge_index	source and destination robot indices for every communication link
Edge features	constant value 1 for each real edge
Batch index	identifies which nodes belong to each graph in a mini-batch

The output is 26 class logits for robustness classes 0 through 25. Softmax converts them into class probabilities. The point prediction is the class with the largest probability.

The saved New GATv2 checkpoint was trained with batch size 32 and reached its best validation loss at epoch 20. On 100 independent test graphs:

Metric	Result
Exact robustness accuracy	88%
Raw prediction ≤ actual robustness	94%
Mean actual − prediction	0.0
Within ±1 class	100%
Within ±2 classes	100%

5. Conformal prediction

CP does not train the GNN and does not generate trajectories. It uses a separate labeled calibration set to transform model outputs into uncertainty-aware prediction sets or lower bounds.

LAC score

$$s(x,y) = 1 - p(y \mid x)$$

LAC returns a set of plausible robustness classes. For planning, the current implementation uses the smallest member of that set as a lower endpoint.

One-sided robustness score

$$s = \hat{r} - r_{actual}$$

A calibrated quantile q gives the lower bound:

$$\underline{r} = \text{clip}(\text{ceil}(\hat{r} - q), 0, 25)$$

At desired coverage 90%, the saved checkpoint produced:

CP result	Observed result
LAC calibration coverage	901/1,000 = 90.1%
LAC test-set coverage	95/100 = 95%
Mean LAC set size	1.18 classes
LAC lower-endpoint coverage	97%
One-sided calibration coverage	924/1,000 = 92.4%

One-sided test lower-bound coverage	94%
One-sided quantile	0

> The calibration data contain no robustness-zero graphs, while the test data do. Also, the one-sided quantile is zero, so the current one-sided “lower bound” equals the GNN point prediction. CP therefore adds no downward correction in the current planner configuration. These results are empirical coverage measurements, not a trajectory-level guarantee.

6. What one SVGD particle means

In SVGD, a particle is one possible solution. Here, the solution is a complete joint trajectory for the entire robot team:

$$Z_k \in R^{T \times N \times 2}$$

where k identifies the candidate trajectory, T is the number of future steps, and N is the number of robots.

Current demo quantity	Value	Meaning
Robots	10	Ten fixed robot identities with fixed starts and goals
Future steps	4	Four future positions for every robot
Candidate trajectories / SVGD particles	16	Sixteen alternative joint trajectories for the same ten robots
SVGD iterations	5	Five simultaneous updates of all sixteen candidates

One particle therefore contains 4 × 10 × 2 = 80 coordinate values. Each particle produces four communication graphs—one graph at every future step. The 16 particles are not different robot teams.

Initial trajectory sampling

For each robot, the code first interpolates a straight line between its fixed start and goal:

$$x^{line}_{t,i} = x_{0,i} + (t/T)(g_i - x_{0,i})$$

Candidate k is produced by adding seeded Gaussian noise:

$$x^{(k)}_{t,i} = x^{line}_{t,i} + \varepsilon^{(k)}_{t,i}$$

7. How the planner turns a trajectory into graph robustness

All robot positions at step t	→	Edge if distance ≤ radius	→	Communication graph G_t
Degrees + edges	→	New GATv2	→	Predicted robustness
Prediction + CP calibration	→	Lower bound $\underline{r}_t$	→	Worst future bound $\min_t \underline{r}_t$

The GNN does not predict the robots' positions. SVGD proposes and updates positions; the GNN estimates the robustness of the graphs induced by those positions.

8. Cost used to update trajectories

Let $x_{t,i}$ be robot i's proposed position at future step t, $x_{0,i}$ its fixed start, g_i its goal, R the communication radius, d_{min} the minimum separation, s_{max} the maximum step, and r_{req} the required robustness.

The full GNN+CP objective is:

$$C(Z) = C_{goal} + C_{motion} + C_{separation} + C_{step} + C_{degree} + C_{CP}$$

Term	Implemented form and purpose
Goal	w_g mean $\|\|x_{T,i} - g_i\|\|^2$; brings final positions toward goals
Motion	w_p mean $\|\|x_{t,i} - x_{t-1,i}\|\|^2$; discourages unnecessary movement
Separation	w_{sep} $[d_{min} - \|\|x_{t,i} - x_{t,j}\|\|]_+^2$; penalizes robots that are too close
Maximum step	w_{step} $[\|\|x_{t,i} - x_{t-1,i}\|\| - s_{max}]_+^2$
Soft degree	w_{degree} $[r_{req} - \tilde{d}_{t,i}]_+^2$; cheap differentiable connectivity heuristic
GNN+CP	w_{CP} $[r_{req} - \min_t \underline{r}(G_t)]_+^2$; penalizes the worst future robustness bound

The differentiable soft degree is:

$$\tilde{d}_{t,i} = \Sigma_{j \neq i} \, \text{sigmoid}((R - \|\|x_{t,i} - x_{t,j}\|\|)/\tau)$$

Graph edges and predicted classes are discrete, so the CP term does not have an ordinary path derivative. The prototype estimates its smoothed gradient using positive and negative random perturbations:

$$\nabla C_{CP}(Z) \approx [C_{CP}(Z+\sigma\varepsilon) - C_{CP}(Z-\sigma\varepsilon)] \, \varepsilon / (2\sigma)$$

The separation and maximum-step terms are penalties, not hard constraints. Small violations can remain in the selected trajectory.

9. Correct SVGD update order

SVGD does not select a trajectory and then apply repulsion. All candidate trajectories remain active until the final selection.

1. Create 16 candidate joint trajectories.
2. Evaluate the cost and cost gradient for all 16 candidates.
3. Calculate attraction toward lower-cost trajectory regions.
4. Calculate repulsion between complete candidate trajectories.
5. Combine both directions and update all 16 candidates simultaneously.
6. Repeat for the specified number of iterations.
7. Only then evaluate final costs and return the lowest-cost candidate.

With target density p(Z) proportional to exp(−C(Z)), the update for candidate i is:

$$Z_i \leftarrow Z_i + \eta/M \, \Sigma_{j=1}^{M}[k(Z_j,Z_i)(-\nabla C(Z_j)) + \nabla_{Z_j} k(Z_j,Z_i)]$$

The first term moves particles toward lower-cost regions. The second maintains diversity between complete team trajectories.

> **Do not mix up two repulsions:** the separation cost repels robots inside one trajectory; SVGD kernel repulsion separates entire candidate trajectories from one another.

10. The three-way ablation

All methods use the same starts, goals, 16 initial candidates, random seed, horizon, and iteration count. Only the planning objective changes.

Method	Cost active during SVGD	Question
simple	Goal, motion, separation, maximum step	What happens without a robustness objective?
soft_degree	Simple costs + smooth degree shortfall	Is a cheap local connectivity heuristic sufficient?
gnn_cp	Soft-degree costs + GATv2/CP robustness shortfall	Does the learned graph-level signal improve selected trajectories?

The simple and soft-degree planners do not query GATv2 while optimizing. After optimization, all selected trajectories are scored by the same GNN+CP evaluator and, for at most 10 robots, the exact exhaustive robustness checker.

11. Current experimental result

Scenario: synthetic 10-robot example, four future steps, 16 candidates, five SVGD iterations, required robustness 2, maximum step 0.20, minimum separation 0.03.

Method	CP bound	Exact minimum r	Goal error	Max step	Min separation	Shared full cost	Time
simple	1	1	0.068	0.200	0.036	5.019	0.333 s
soft_degree	0	0	0.067	0.248	0.018	20.007	0.289 s
gnn_cp	2	2	0.059	0.201	0.035	0.003	2.276 s

Interpretation:

- The simple trajectory did not meet the placeholder robustness target: exact minimum r = 1.
- The soft-degree heuristic performed poorly in this run: exact minimum r = 0 and both soft physical limits were violated.
- GNN+CP met the placeholder robustness target with exact minimum r = 2 and had the smallest goal error.
- GNN+CP was about 6.8× slower than the simple method because of repeated graph construction, neural inference, CP scoring, and random gradient probes.
- The GNN+CP maximum step was 0.201 versus a soft limit of 0.200, showing why hard constraints may be needed.

“Shared full cost” evaluates all selected trajectories with the same full objective, including the CP penalty. It is not merely each method’s private optimization objective.

> This is one scenario and one seed. It supports the claim that integration works, not the claim that GNN+CP is generally superior. The soft-degree result could change with weights, iterations, and initialization.

12. Robustness threshold and the role of F

The demo currently contains `required_robustness: 2`. That value was selected only to create a nontrivial synthetic example.

If the research goal is standard W-MSR resilient consensus under an F-local adversary model, use:

$$r_{\text{required}} = 2F + 1$$

Maximum local adversaries F	Scalar robustness target 2F+1
0	1
1	3
2	5
3	7

The scalar 2F+1 rule is model-dependent. F-total, asynchronous, local-broadcast, and other consensus settings can use refined conditions such as (r,s)-robustness. The current GNN predicts scalar r-robustness only. Before changing the code, confirm the adversary model and consensus protocol with the professor.

13. What “prototype” means here

The integration is runnable, but the following parts are not yet tied to a physical robot system:

- the ten starts and goals are synthetic;
- coordinates use arbitrary units;
- the distance-threshold communication model is assumed rather than measured;
- there are no obstacles or full robot dynamics;
- movement and separation are soft penalties rather than enforced constraints;
- the planner runs once as a trajectory optimizer; it is not yet an MPC loop;
- the exact robustness checker is restricted to at most ten robots because exhaustive subset enumeration is expensive;
- adaptive selection among many queried graphs weakens a direct interpretation of ordinary split-CP coverage;
- the calibration distribution does not cover robustness zero.

“Prototype” means the component connection has been demonstrated. It does not mean the selected trajectory is ready for execution on robots.

14. Where to change the experiment

Quantity	Where it is set	How to change it
Number of robots	Length of start and goal arrays in planning/examples/demo_scenario.json	Add or remove matching start/goal pairs; there is no --robots flag
Start and goal positions	start and goal arrays in the scenario JSON	Replace with real positions in consistent units
Communication radius	config.communication_radius	Use a measured or modeled communication rule
Minimum separation	config.minimum_separation	Set from robot geometry and safety margin
Maximum step	config.maximum_step	Set from time step and motion capability
Robustness threshold	config.required_robustness	Derive from F and the chosen consensus/adversary model
Alternative joint trajectories	--particles, default 16	Increase for exploration; runtime rises
Future horizon	--steps, default 4	More steps create a longer trajectory and more graph queries
SVGD updates	--iterations, default 5	Increase for more optimization; runtime rises
Initialization diversity	--noise, default 0.07	Controls noise around straight-line trajectories

15. Commands for the meeting

Verify GPU

```
nvidia-smi
JAX_PLATFORMS=cuda .venv/bin/python -c 'import jax; print(jax.devices())'
```

Evaluate New GATv2 and both CP scores

```
JAX_PLATFORMS=cuda .venv/bin/python jax_test_new_gatv2.py \
  --checkpoint checkpoints/full_run_xK0R3I/new_gatv2.msgpack \
  --batch-size 32 \
  --conformal-coverage 0.9
```

Run the three-way trajectory comparison

```
JAX_PLATFORMS=cuda .venv/bin/python run_svgd_experiment.py \
  --scenario planning/examples/demo_scenario.json \
  --checkpoint checkpoints/full_run_xK0R3I/new_gatv2.msgpack \
  --particles 16 \
  --steps 4 \
  --iterations 5 \
  --output svgd_three_way_results.json
```

Create a custom scenario

```
cp planning/examples/demo_scenario.json planning/examples/my_scenario.json

# Edit my_scenario.json, then:
JAX_PLATFORMS=cuda .venv/bin/python run_svgd_experiment.py \
  --scenario planning/examples/my_scenario.json \
  --particles 16 --steps 4 --iterations 20 \
  --checkpoint checkpoints/full_run_xK0R3I/new_gatv2.msgpack \
  --output my_svgd_results.json
```

16. Recommended next experiment

1. **Confirm the theory:** ask which adversary model, consensus protocol, and F value the professor intends. Derive the correct robustness target.
2. **Confirm graph generation:** find how training graphs were produced and determine whether a physical distance-radius graph matches that distribution.
3. **Define physical scenarios:** real robot count, starts, goals, time step, communication rule, speed, separation, obstacles, and dynamics.
4. **Run an ablation matrix:** simple, soft degree, and GNN+CP across multiple scenarios and at least 10–20 random seeds.
5. **Report distributions:** success rate for exact r ≥ threshold, minimum exact r, constraint violations, goal error, mean runtime, and worst-case runtime.
6. **Sweep planner settings:** candidate count, horizon, iterations, CP weight, and robustness threshold.

7. **Decide whether to keep SVGD:** if success or runtime is poor after reasonable tuning, implement the same three-way objectives in MPPI or MBD.
8. **Move toward MPC only after that:** repeatedly observe, plan over a horizon, execute the first control step, and replan.

17. Suggested presentation script

Opening: “The goal is to use a learned graph-robustness estimate inside a multi-robot trajectory planner. The GNN does not generate trajectories. SVGD generates and updates candidate joint trajectories; each future robot configuration becomes a graph, and GATv2 plus CP scores its robustness.”

Dataset result: “I replaced global per-graph padding with compact storage and controlled batch-level padding for JIT reuse. In a matched GATv2 run this reduced steady epoch time from roughly 94 seconds to 16 seconds, although test accuracy changed from 93% to 88%, so the representation tradeoff needs further investigation.”

Planner result: “I compared the same initial SVGD trajectories under three objectives. In one synthetic scenario, the GNN+CP objective found exact minimum robustness 2, versus 1 for the simple cost and 0 for soft degree, but required about 2.28 seconds rather than 0.33 seconds.”

Limit: “This is an integration result, not yet a physical or statistical validation. The scenario parameters are synthetic, the one-sided CP correction is zero, and the target r = 2 is a placeholder.”

Decision request: “I need to confirm the intended adversary model and F, how the training graphs were generated, and the physical robot constraints. Then I can run the three-way comparison across realistic scenarios and decide whether SVGD is strong and fast enough or whether to move to MPPI or MBD.”

18. Likely questions and direct answers

Question	Answer
What is a particle?	One complete joint future trajectory for all robots. Sixteen particles mean sixteen alternatives for the same team.
Does SVGD operate on nodes or graphs?	Neither directly. It updates all waypoint coordinates in a joint trajectory. Those positions induce one graph per future step.
What does the GNN predict?	One scalar robustness class for the whole graph at each future step.
Why train a GNN if positions are known?	Positions determine the graph, but exact r-robustness is combinatorial. The GNN is intended as a fast approximation during many planning queries.
Do we still need	Yes for offline training. Planning with an existing checkpoint does not reload the

training_data.json?	training set.
Is CP a guarantee here?	No. It is a calibrated marginal uncertainty estimate; distribution shift and adaptive trajectory selection prevent a direct selected-plan guarantee.
Why 10 robots?	It was selected for the synthetic demo because the included exact checker is feasible up to ten robots.
Why 16 candidates?	It is a manageable prototype setting, not a theoretically optimal number.
Why is GNN+CP slower?	It builds and scores many future graphs and estimates a gradient for a discrete robustness penalty using multiple probes.
Is this MPC?	No. It is currently a one-shot trajectory optimizer. MPC would execute only the first step, observe the system again, and replan repeatedly.
Should the target be 2F+1?	Yes for the standard scalar r-robustness sufficiency condition under the corresponding W-MSR/F-local assumptions. Confirm the exact system model first.

19. Code map

Path	Purpose
gnn/variants.py	Compact records, batch-level padding, training, testing, and checkpoints
gnn/models/gatv2.py	Current New GATv2 graph classifier
conformal/scores.py	LAC and one-sided conformal scores
planning/examples/ demo_scenario.json	Synthetic robot starts, goals, and planner settings
planning/run_experiment.py	Initial trajectories, three-way experiment, exact check, and reporting
planning/svgd_cp_planner.py	Graph construction, GATv2/CP scorer, costs, gradient approximation, and SVGD update
planning/README.md	Planner usage and limitations
legacy/	Historical padded pipeline and older implementations

References

1. H. J. LeBlanc, H. Zhang, X. Koutsoukos, and S. Sundaram, “Resilient Asymptotic Consensus in Robust Networks,” IEEE Journal on Selected Areas in Communications, 2013.
2. H. LeBlanc et al., “Resilient Asymptotic Consensus in Asynchronous Robust Networks,” Allerton Conference, 2012.
3. Q. Liu and D. Wang, “Stein Variational Gradient Descent: A General Purpose Bayesian Inference Algorithm,” NeurIPS, 2016.

This brief records the current repository behavior and measured results as of 15 October 2026. Synthetic scenario results should not be interpreted as robot-system guarantees.